

İZMİR KATİP ÇELEBİ UNIVERSITY
FACULTY OF ENGINEERING AND ARCHITECTURE
DEPARTMENT OF COMPUTER ENGINEERING

CENG 454: REINFORCEMENT LEARNING

FINAL PROJECT REPORT:
CardShark-RL

Reinforcement Learning of Behavioral Tells in 5-Card Draw Poker

Student Name: Sarp Sünbül
Student Number: 230401114
Student Name: Alkım Gönenç Efe
Student Number: 220401050
Student Name: Damla Parlakyıldız
Student Number: 220401067

Submission Date:
May 22, 2026

Reinforcement Learning of Behavioral Tells in 5-Card Draw Poker

Alkım Gönenç Efe¹, Sarp Sünbül², Damla Parlakyıldız³

Department of Computer Engineering

Izmir Katip Celebi University

Izmir, Turkey

Abstract—We present CardShark-RL, a reinforcement learning system for heads-up limit 5-Card Draw Poker that compares two opponent-adaptation strategies: explicit modeling, where a one-hot archetype identifier is fed directly into the agent’s observation, and implicit modeling, where the agent works from a rolling window of 10 behavioral statistics and never sees an opponent label. Both agents are trained with MaskablePPO [1] in a custom PettingZoo [2] environment, with Optuna-based hyperparameter search [3] and hand-crafted reward shaping. Evaluated over 10,000 hands each against three scripted archetypes—Calling Station, Maniac, and Rock—the explicit agent (Model A) averages +180.65 BB/100, while the implicit agent (Model B) averages +166.50 BB/100 ($p = 0.029$). The gap is real but modest. More interesting: in a 1,500-hand non-stationary tournament where the opponent switches archetype mid-session, Model B (+160.35 BB/100) edges out an oracle-labeled explicit agent (+156.28 BB/100). We analyze the behavioral mechanism through which Model B exploits opponent draw-count patterns without ever knowing who it is playing.

Index Terms—reinforcement learning, opponent modeling, poker AI, MaskablePPO, imperfect-information games, reward shaping, behavioral inference, hyperparameter optimization

I. INTRODUCTION

Poker forces an agent to reason under hidden information and adapt to an opponent whose strategy it cannot observe directly. The combination of imperfect information, sequential decisions, and an adversarial opponent makes it a useful benchmark, because the skills it demands transfer to real decision problems.

Systems like Libratus [4], DeepStack [5], and Pluribus [6] have solved large variants of Texas Hold’em by minimizing exploitability via CFR [7].

They are robust against any opponent. What they are not designed to do is actively exploit a weak one. An equilibrium strategy leaves money on the table against predictable opponents; exploitative play picks it up.

We built CardShark-RL to investigate exploitative RL in 5-Card Draw, a variant with a clean information structure (each player draws once, revealing something about their hand) and enough strategic depth to be interesting. The central question is whether an agent can learn to profit from opponent behavioral patterns using only indirect evidence. No archetype label, just the history of what the opponent has done.

The system implements two agents. Model A gets the opponent’s archetype fed to it as a one-hot vector; it knows exactly who it is playing. Model B gets a rolling window of 10 behavioral statistics and has to figure the rest out. We run Optuna-based hyperparameter search with a Rock-weighted objective (to avoid agents that just farm easy wins), use reward shaping to push toward aggression, and run a behavioral matrix analysis, which is Model B’s post-draw action distribution conditioned on opponent draw count, to understand what implicit information it actually learned to use.

II. RELATED WORK

A. Poker AI

CFR-based systems have dominated competitive poker AI since Polaris [7]. Libratus [4] and DeepStack [5] both solved heads-up no-limit Hold’em at a superhuman level in 2017 through different approaches to endgame solving [8]. Pluribus [6] extended this to six-player play. All of these are equilibrium-seeking; they are hard to beat, but they do not specifically target opponent weaknesses.

Our work sits in a different space. We train explicitly to exploit fixed behavioral patterns, accepting that the resulting policy would be vulnerable against an adversary that adapts in return.

B. Action Masking

In 5-Card Draw, the agent has 35 possible actions but only a small subset are legal at any step. Huang and Ontañón [9] show that hard action masking—setting illegal action log-probabilities to $-\infty$ before the softmax—is both theoretically sound and empirically far better than penalty-based alternatives when invalid actions are frequent. We use MaskablePPO [1] for exactly this reason.

C. Opponent Modeling

There is a large literature on opponent modeling in multi-agent RL, ranging from Bayesian type inference to neural latent embeddings. The rolling-statistics approach we use for Model B is conceptually similar to behavioral profiling in poker theory: track VPIP, Pre-draw Raise frequency, Aggression Factor, and draw tendencies over a window of recent hands, and let the policy condition on those statistics rather than on an explicit type label.

D. Reward Shaping and HPO

Ng et al. [10] established that potential-based reward shaping preserves optimal policies. We use two auxiliary shaping terms to guide the agent toward aggressive exploitation. Hyperparameter search uses Optuna [3] with TPE sampling and median pruning over a 15-dimensional space.

III. ENVIRONMENT

A. Game Rules

CardShark-RL implements limit heads-up 5-Card Draw. Both players post a 1-chip ante. Pre-draw betting uses 2-chip bets (up to 4 raises). Each player discards and draws 0–5 cards. Post-draw betting uses 4-chip bets (up to 4 raises), followed by showdown. A player can win before showdown by making the opponent fold.

B. Environment Implementation

The game runs as a PettingZoo AEC environment [2], wrapped by a Gymnasium-compatible [11] single-agent interface (DrawPokerGymEnv) that auto-steps the opponent and exposes a standard

(obs, reward, done, info) tuple to the RL agent (player_0).

C. Action Space

The action space is `Discrete(35)`: Fold (0), Call/Check (1), Raise/Bet (2), and 32 draw actions (3–34) encoding a 5-bit bitmask over card positions. During betting phases only actions 0–2 are valid; during draw phases only 3–34 are. Action masking enforces this at every step.

D. Observations

Both models share 23 base features: 10 for card encoding (normalized rank and suit per card), 2 for hand strength (category and evaluation score), 3 for pot state (size, bet-to-call, pot odds), 3 phase indicators, 1 for opponent draw count (or -1 if pre-draw), and 4 strategic features (position, own raise count, opponent aggression flag, opponent pre-draw raise flag).

Model A appends a 3-dimensional one-hot opponent archetype vector, giving a 26-dimensional observation.

Model B appends a 10-dimensional rolling statistics vector computed over the last W hands: fold frequencies pre- and post-draw, raise frequency post-draw, average cards drawn, raise-then-stand-pat correlation, fold-after-drawing-3 frequency, raise-after-standing-pat frequency, VPIP, PFR, and Aggression Factor. Total: 33 dimensions.

E. Hand Evaluator

Cards are encoded as integers 0–51 where rank = $c \bmod 13$ and suit = $\lfloor c/13 \rfloor$. Hand strength is a scalar = category $\times 10^6$ + tiebreaker, with the tiebreaker using base-14 rank encoding. This gives a total ordering across all nine hand categories.

F. Opponent Archetypes

We wrote three scripted opponents:

CallingStation never raises and never folds. It draws optimally (keeps best group, discards kickers). A completely passive opponent.

Maniac raises 70% of the time when it can, folds only 5% of the time, and chases flush/straight draws 60% of the time by drawing one card.

Rock plays a tight range: folds unpaired hands pre-draw to any bet, calls with a pair below Jacks,

raises with Jacks-or-better, and stands pat on two-pair or better. It is the hardest to exploit because it rarely puts chips in without a strong hand.

IV. METHODOLOGY

A. MaskablePPO

Both agents use MaskablePPO [1], a PPO variant that applies action masking directly in the policy update. The clipped objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (1)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and $\hat{A}_t$ is the GAE [12]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

Illegal actions are masked by setting their log-probabilities to $-\infty$ before the softmax.

B. Network Architecture

Both models use a separate actor and critic MLP with no shared parameters. Model B (implicit) uses four hidden layers of 256 neurons each, chosen by HPO. Model A (explicit) uses three layers of 256 neurons.

C. Reward Function

The base reward is net chip profit: $r = \text{pot_won} - \text{chips_invested}$ for the winner, $r = -\text{chips_invested}$ for the loser. Two shaping terms are added [10]: a fold penalty ($-\alpha$) when the agent folds in a spot where checking was free (bet-to-call = 0), and a steal bonus ($+\beta$) when the agent raises and the opponent folds. The weights are tuned by Optuna; for Model B, $\alpha = 0.296$ and $\beta = 0.448$.

D. Opponent Scheduling

Model A uses random scheduling, meaning a new archetype is drawn uniformly before each episode. Since it sees the archetype label directly, it can condition immediately.

Model B uses a hybrid schedule. The first H episodes use block scheduling: opponents cycle in blocks of B consecutive hands, giving the rolling statistics time to build a stable signal before the agent tries to exploit it. After H episodes, scheduling reverts to random. H and B are HPO parameters.

E. Hyperparameter Search

We ran Optuna [3] with TPE sampling and MedianPruner over a 15-dimensional space. Each trial trains for 300,000 timesteps and is scored by a Rock-weighted objective:

$$\text{score} = 0.5 \cdot \text{BB}_{100}^{\text{Rock}} + 0.25 \cdot \text{BB}_{100}^{\text{Station}} + 0.25 \cdot \text{BB}_{100}^{\text{Maniac}} \quad (3)$$

The Rock weight is doubled because farming the Maniac or CallingStation is easy; good performance against the Rock is evidence that the agent learned something real. The best configurations are in Table I.

TABLE I
BEST HYPERPARAMETERS FOUND BY OPTUNA

Hyperparameter	Model A (Expl.)	Model B (Impl.)
learning_rate	1.63×10^{-4}	3.50×10^{-4}
n_steps	4096	2048
batch_size	32	64
n_epochs	22	11
gamma	0.9658	0.9894
ent_coef	0.00103	0.00457
clip_range	0.242	0.194
gae_lambda	0.874	0.922
vf_coef	0.475	0.430
max_grad_norm	0.484	0.464
fold_penalty	0.379	0.328
steal_bonus	0.492	0.212
lr_schedule	constant	linear
net_arch	$[128] \times 2$	$[256] \times 4$
rolling window W	N/A	50
block size B	N/A	400
hybrid switch H	N/A	972k steps

F. Training and Evaluation

Both models train for 1,000,000 timesteps using 4 parallel environments (DummyVecEnv). A per-opponent callback evaluates BB/100 every 50,000 timesteps to track learning dynamics.

Performance is reported in *big blinds per 100 hands* (BB/100):

$$\text{BB/100} = \frac{\text{total profit}}{\text{number of hands}} \times 100 \quad (4)$$

where 1 BB = 4 chips. Final evaluation runs 10,000 hands against each fixed archetype over 3 independent seeds. Model B uses a 50-hand random warm-up to populate the statistics buffer before evaluation starts.

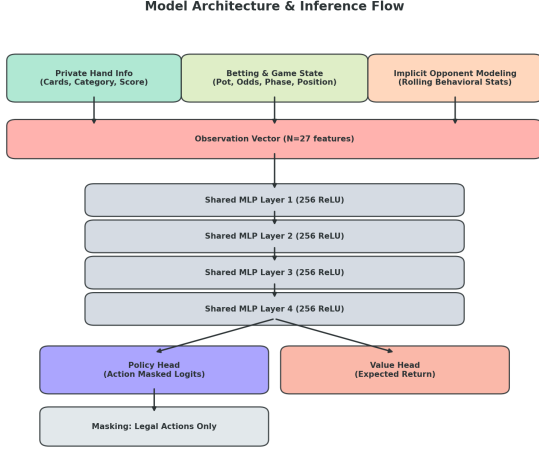


Fig. 1. Model B’s inference path: hand features, game state, and the 10-dimensional rolling statistics buffer feed four 256-unit MLP layers, which split into policy and value heads. Action masking zeroes out illegal actions before sampling. The rolling buffer is the only opponent-identity signal available.

V. RESULTS

A. Cross-Evaluation

Table II summarizes BB/100 across three seeds per opponent. Both trained agents are strongly profitable against all three archetypes; the random baseline loses to all of them.

TABLE II
BB/100 BY MODEL AND OPPONENT (MEAN $\pm$ STD)

Opponent	Model A	Model B	Random
Calling St.	+86.5 $\pm$ 4.3	+79.9 $\pm$ 5.0	-143.8 $\pm$ 2.7
Maniac	+408.0 $\pm$ 15.7	+390.8 $\pm$ 11.5	-222.1 $\pm$ 0.1
Rock	+47.5 $\pm$ 2.8	+28.9 $\pm$ 6.5	-151.3 $\pm$ 2.0
Average	+180.65	+166.50	-172.44

Model A leads by about 14 BB/100 overall. Against the Rock the gap is largest (+18.6 BB/100); the Rock’s tight range is hard to read without knowing its archetype, and the rolling statistics take longer to build a usable signal. Against the Maniac the difference narrows to +17.2 BB/100. The Maniac’s high VPIP and frequent single-card draws are distinctive enough that Model B picks up the pattern quickly within the $W = 50$ window.

Fig. 2 shows the profitability breakdown. Fig. 3 shows cumulative profit over 30,000 hands (three seeds concatenated): both trained agents grow lin-

early, confirming that the exploitative advantage does not erode over time.

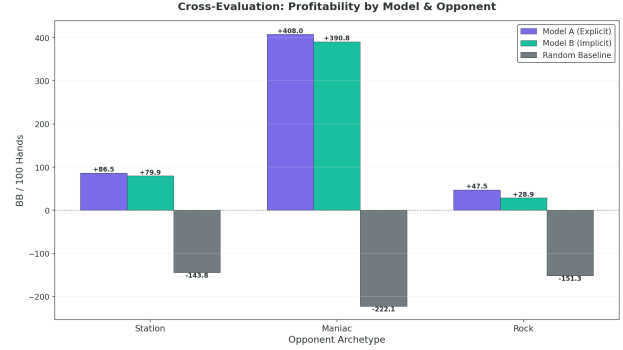


Fig. 2. Explicit identity gives a clear edge against the Rock but matters less against the Maniac, where behavioral patterns are obvious from draws alone. Model B still comfortably beats all three.

B. “The Tell”: Behavioral Matrix Analysis

The most illuminating result is not the aggregate BB/100 but what Model B actually learned to do. Fig. 4 shows Model B’s post-draw action distribution (Fold / Call / Raise) broken down by opponent draw count (0, 1, or 3 cards) for each archetype.

Against the CallingStation, the pattern is flat: Model B calls or raises regardless of draw count (0% fold rate at draw 0 or draw 1). There is nothing to read in a passive opponent’s draws.

Against the Maniac, a single-card draw lifts Model B’s raise frequency to 52%. A loose opponent drawing one card is typically chasing a flush or straight that usually misses, making it a good spot to apply pressure.

The sharpest pattern is against the Rock. When the Rock stands pat (draw 0), Model B responds with 50% call and 43% raise. It treats this as a strong-hand signal and stays in. When the Rock draws one card, Model B’s call and raise both drop to 0%. Drawing one is a rare Rock action that, historically in training, correlated with high-value draws. The agent learned to get out of the way. These are opponent-specific, draw-count-driven heuristics, and Model B discovered them without ever seeing an archetype label.

C. Non-Stationary Adaptation

We ran a 1,500-hand tournament in which the opponent switches silently from CallingStation (hands

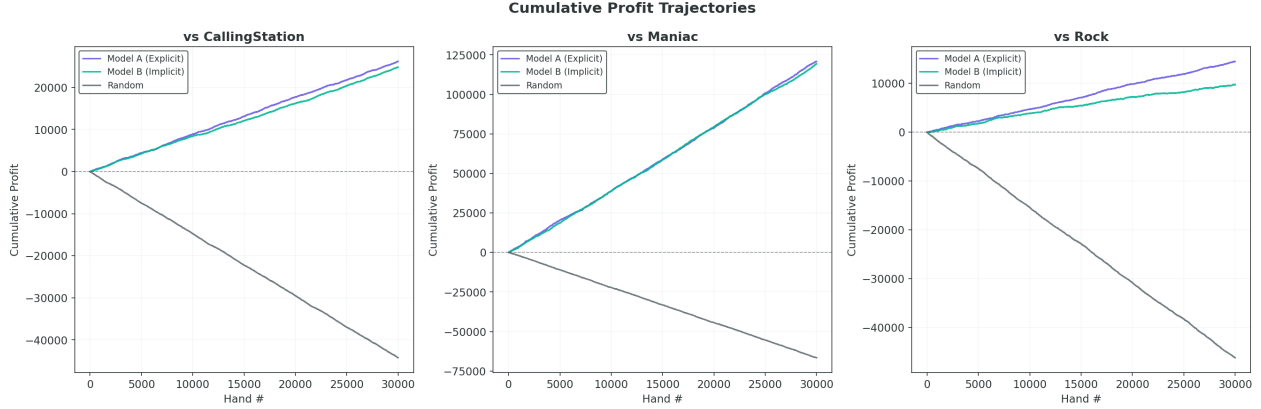


Fig. 3. Cumulative profit grows linearly for both trained agents across 30,000 hands—the exploitative advantage is stable and does not saturate. The random baseline loses at a near-constant rate, confirming the archetypes are consistently exploitable.

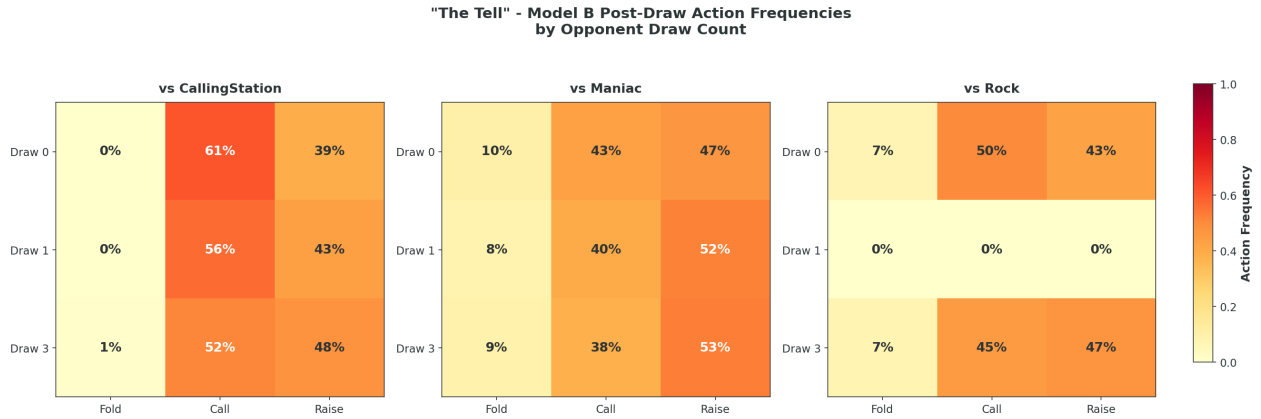


Fig. 4. Model B reads opponent draw counts as hand-strength signals: against the Rock, standing pat triggers cautious call/raise (50/43%); drawing one card collapses both to 0%, as that is an anomalous Rock action historically correlated with strong draws. Against the CallingStation, draw count barely changes anything—it is always safe to bet.

1–500) to Maniac (501–1,000) to Rock (1,001–1,500) without resetting Model B’s statistics buffer. Two Model A variants serve as baselines: an *Oracle* that receives the correct archetype label at each switch, and a *Blind* variant stuck with one wrong label throughout.

Results are in Table III. Model B leads at +160.35 BB/100, edging the Oracle at +156.28 BB/100. The Blind variant falls to +50.01 BB/100, confirming that the explicit label is only useful when it is correct. Fig. 5 shows the rolling statistics tracking each opponent transition within ~ 50 –100 hands. Fig. 6 shows the corresponding win-rate curve with brief transients at each switch.

TABLE III
NON-STATIONARY TOURNAMENT RESULTS (1,500 HANDS)

Agent	BB/100
Model B (Implicit)	+160.35
Model A (Explicit – Oracle)	+156.28
Model A (Explicit – Blind)	+50.01

D. Statistical Validation

We pooled 270,000 hands (3 opponents $\times$ 3 seeds $\times$ 10,000 hands). A paired t -test on per-hand profit gives $t = -2.183$, $p = 0.029$: the Model A advantage is significant at $\alpha = 0.05$. Model B vs. the random baseline is not close ($t = +106.85$, $p \approx 0$). Model B’s 95% confidence interval is +1.71

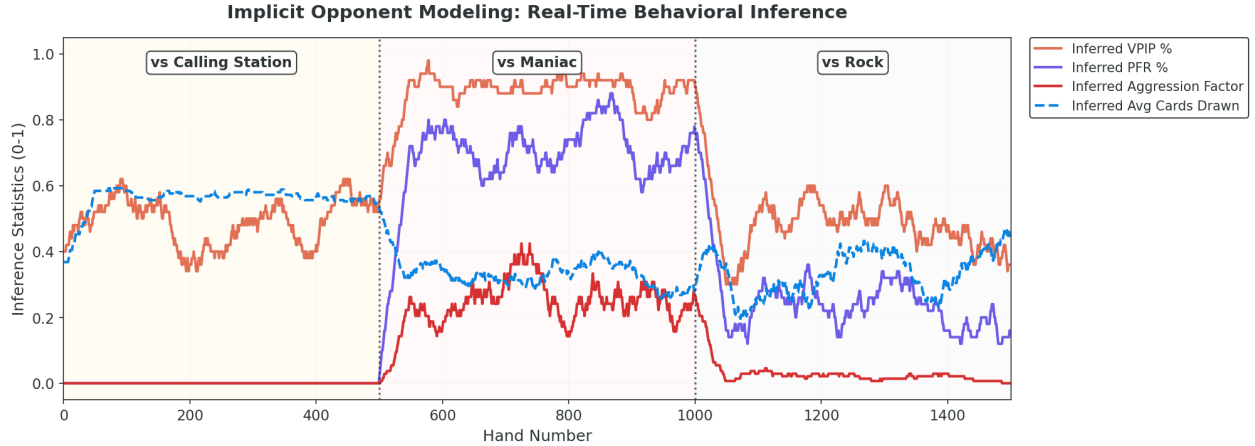


Fig. 5. Rolling VIP, PFR, Aggression Factor, and Avg. Cards Drawn shift noticeably within 50–100 hands of each opponent switch. The buffer does not reset, so the transition is gradual, which can be an advantage or a limitation depending on how fast the opponent changes.

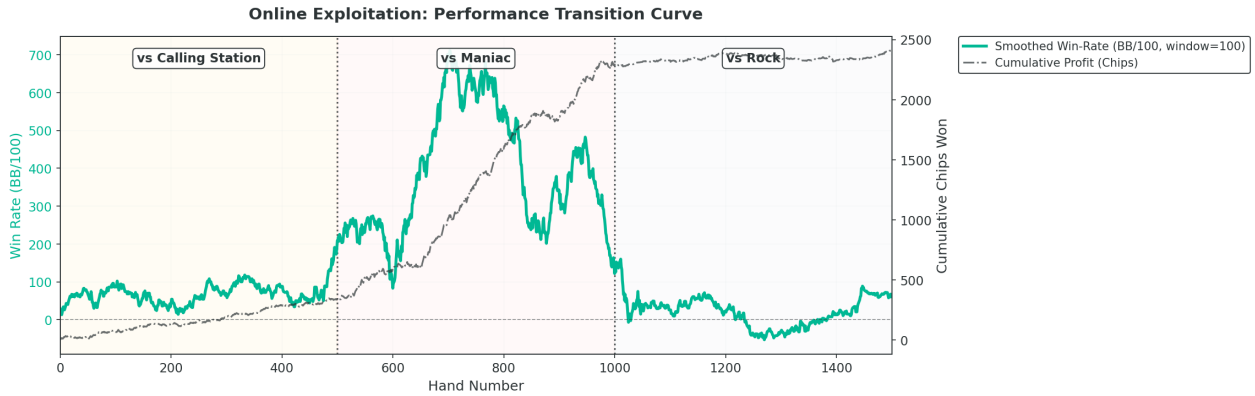


Fig. 6. Win rate (BB/100, smoothed over 100 hands) peaks during the Maniac segment, drops briefly at each switch, then recovers. The recovery time—roughly 50–100 hands—matches how quickly the rolling statistics stabilize.

chips/hand [1.65, 1.76], corresponding to +170.87 BB/100 [165.31, 176.44].

VI. DISCUSSION

A. How Much Does the Label Actually Help?

About 14 BB/100 on average, concentrated almost entirely in the Rock matchup. The Rock emits sparse behavioral signals. It folds a lot, plays few hands, and rarely deviates from its pattern. A rolling window of 50 hands captures some of this, but not as cleanly as a direct label. Against the Maniac, the label advantage nearly disappears, because the Maniac’s draws and aggression are distinctive enough that 50 hands is plenty of data.

This suggests a diminishing-returns effect: implicit modeling closes most of the gap when the

opponent is behaviorally distinctive, but struggles against opponents whose signal is infrequent or weak.

B. Why Model B Beats the Oracle in Non-Stationary Play

Model B’s rolling statistics give it a continuous, graded representation of opponent behavior. When the opponent switches archetypes, the buffer does not snap to the new label, it gradually shifts as new evidence accumulates. This is actually useful during the transition window, when the opponent’s behavior is a mixture of old and new patterns. The Oracle gets the correct label immediately, but the policy was trained on clean archetypes, not mixtures; Model B’s gradual buffer update may

better match the policy’s training distribution near switch points.

The behavioral matrix in Fig. 4 supports this: Model B is already conditioning on fine-grained draw-count statistics rather than on a discrete type, so partial evidence is still usable.

C. Limitations

All three opponents are fixed-strategy scripts. The agent has never played an adaptive opponent, a self-play adversary, or a human. Against someone who notices they are being exploited and adjusts, Model B’s strategy would need to update faster than the rolling window allows.

The limit betting structure removes stack-depth considerations that dominate no-limit play. The cold-start problem—the $W = 50$ window needs several dozen hands before it stabilizes—also means Model B is handicapped at the start of a new session or right after an opponent switches.

Heads-up only. Multiplayer would require rethinking the observation design substantially for optimal play.

VII. CONCLUSION

We built a complete RL pipeline for 5-Card Draw Poker and used it to test whether an agent can exploit behaviorally distinct opponents without knowing which one it is facing. It can, at a cost of about 14 BB/100 versus an agent that gets a direct label. The gap is real and statistically significant, but modest enough to suggest that draw-count telemetry carries most of the relevant information.

The behavioral matrix analysis is the clearest evidence of what the agent actually learned. Model B treats a Rock standing pat as a “stay in” signal and a Rock drawing one card as a “get out” signal, with no explicit hand-range model and no archetype identifier. That is the Tell. The agent discovered it from reward alone.

In non-stationary play, the implicit agent outperforms an oracle-labeled explicit agent, which is the more practically useful result: real opponents do not come with labels, and they change.

Future directions include no-limit betting, self-play training, and combining the rolling statistics with Bayesian hand-range inference to address the Rock’s sparse behavioral signal.

REFERENCES

- [1] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [2] J. K. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, C. Dieffendahl, C. Horsch, R. Perez-Vicente, N. L. Williams, Y. Lokesh, and P. Ravi, “Pettingzoo: Gym for multi-agent reinforcement learning,” *arXiv preprint arXiv:2009.14471*, 2021. [Online]. Available: <https://arxiv.org/abs/2009.14471>
- [3] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [4] N. Brown and T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science*, vol. 359, no. 6374, pp. 418–424, 2018.
- [5] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, and M. Bowling, “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker,” *Science*, vol. 356, no. 6337, pp. 508–513, 2017.
- [6] N. Brown and T. Sandholm, “Superhuman AI for multiplayer poker,” *Science*, vol. 365, no. 6456, pp. 885–890, 2019.
- [7] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, “Regret minimization in games with incomplete information,” in *Advances in Neural Information Processing Systems*, vol. 20, 2007.
- [8] S. Ganzfried and T. Sandholm, “Endgame solving in large imperfect-information games,” in *Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems*, 2015, pp. 37–45.
- [9] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” in *Proceedings of the 35th International FLAIRS Conference*, 2022.
- [10] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proceedings of the Sixteenth International Conference on Machine Learning*, 1999, pp. 278–287.
- [11] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “Openai gym,” 2016.
- [12] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *International Conference on Learning Representations*, 2016. [Online]. Available: <https://arxiv.org/abs/1506.02438>